PROJECT HANDBOOK

Student B — Model Development

Satellite-Based Early Warning for Glacial Lake Outburst Floods (GLOF) · Major Project — 7th Semester · 14-week (3.5-month) week-by-week guide

> **Your job in one line:** teach the computer what a lake normally looks like over the years, so it can notice when a lake starts behaving strangely. You are not predicting disasters — you are spotting 'this is not normal for this lake'.

1. Before you start — set up in the first two days

- Create accounts for **Google Colab**, **GitHub** and **Google Earth Engine** (you will need to read data, even if A does the downloading).
- Set up the same conda environment as Student A so your results run on their data without version clashes.
- Read the **GLOFNet paper** (link in the resource pack) — especially the 'limitations and future work' section. That section is where our project's novelty comes from.
- Make your folder in the shared repository, e.g. */models*, and keep every experiment in a numbered notebook.

2. Your toolkit — what to use and how to use it properly

Tool	What you use it for	How to use it properly
Google Colab	Training and testing your models for free.	Free GPU time is limited — use it only for the deep learning parts, and run the simple models on CPU. Save every trained model to Drive immediately.
pandas + xarray	Turning the raw data cube into a neat table of features (one row per lake per date).	Keep one clear table: lake ID, date, and one column per signal. Every later step reads this one table — do not create five different versions of it.
scikit-learn	The main anomaly-detection models: Isolation Forest, One-Class SVM, PCA.	Fit the model only on *normal* periods, then score everything. Always set random_state so your results can be repeated exactly.
statsmodels / tsfresh	Finding the seasonal pattern in each signal and extracting time-based features.	Glacial lakes have a strong summer/winter cycle. Remove the seasonal pattern first, otherwise every summer looks like an 'anomaly'.
PyTorch	The temporal model — an LSTM or autoencoder that predicts the next value and flags large surprises.	Start small: one lake, one signal. Only scale up once it beats the simple baseline. If it does not beat the baseline, say so honestly — that is a valid result.
MLflow	Recording every experiment: settings used and scores obtained.	Log every run, even failed ones. In month 4 you will need to explain why you chose the final model — MLflow is your evidence.
matplotlib	Plotting signals over time to see what the model sees.	Plot before you model. Most modelling mistakes are visible in a simple line chart.

3. Your week-by-week plan

Fourteen weeks — three and a half months. Each row is one week. If a week slips, tell the team in the stand-up the same week — do not quietly carry it forward.

Month 1 — Understand the data before modelling anything

Week	Focus	What to do	Done when…
Week 1	**Read and set up**	• Read the GLOFNet paper and note its four stated limitations. • Set up your environment and repository folder. • Reproduce the basic numbers reported in the paper (how many images, how many records) as a sanity check. **Tools:** Colab, pandas, GitHub	Environment ready; paper understood
Week 2	**Look at the data**	• Plot glacier speed, temperature and lake size over the years for a few lakes. • Note the seasonal pattern — what happens every summer and every winter. • Write down, in plain words, what 'normal behaviour' looks like for a lake. **Tools:** pandas, matplotlib, xarray	First set of charts and observations
Week 3	**Decide the signals**	• Define exactly what you will measure: lake area, glacier speed and its acceleration, a melt-season temperature index, and moraine slope. • Prototype the lake-area calculation from satellite images using the water index (NDWI). **Tools:** NumPy, rasterio, NDWI formula	Signal definitions agreed with C
Week 4	**Build the feature table**	• Write code that turns A's data cube into one clean table: one row per lake per date. • Agree the exact column names with C so their testing code matches yours. • Handle missing values by marking them, not deleting the row. **Tools:** pandas, xarray	Feature table version 1

Month 2 — Simple methods first

Week	Focus	What to do	Done when…
Week 5	**Simple baselines**	• Build the simplest possible detector: flag a lake when a signal is unusually far from its own average (a z-score). • Remove the seasonal pattern before scoring, so summer is not always flagged. **Tools:** statsmodels, scikit-learn, pandas	Baseline detector running
Week 6	**End-to-end on one lake**	• Run the full chain — data in, score out — for one lake. • Hand the scores to C so they can start building the testing setup. • Do not move on until this simple version genuinely works. **Tools:** Python, pandas	First anomaly scores given to C
Week 7	**Proper anomaly models**	• Train **Isolation Forest** and **One-Class SVM** on the feature table. • Normalise each lake separately — a big lake and a small lake should not be compared directly. • Compare against the simple baseline from week 5. **Tools:** scikit-learn, MLflow	Two anomaly models working
Week 8	**Scores for every lake**	• Produce scores for all 15–25 lakes across the whole time period. • Log all settings in MLflow so the run can be repeated. • Show your charts at the mid-project review. **Tools:** scikit-learn, MLflow, matplotlib	Scores for all lakes; review passed

Month 3 — Deeper models and combining the signals

Week	Focus	What to do	Done when…
Week 9	**Time-aware model**	• Build a model that learns the normal rhythm of a lake and predicts the next value. • Flag the weeks where reality is far from the prediction. • Compare fairly against the simple baseline — if it is not better, report that honestly. **Tools:** PyTorch (LSTM / autoencoder), Colab GPU	Temporal model trained
Week 10	**Better temperature, plus radar**	• Temperature data is very coarse (1 km per pixel) while lakes are small — use height and terrain to estimate a finer value, and check it against the raw values. • Take A's radar data and turn it into signals: change in radar brightness, and lake extent measured by radar. • Add both to the feature table and retrain. **Tools:** scikit-learn regression, DEM data, pandas	Finer temperature and radar signals in the model
Week 11	**Combine into one score**	• Merge all signals into one simple risk number per lake per date. • Add the rule: if too much data is missing, output **'cannot assess'** instead of a low risk score. Never report 'safe' when you simply cannot see. **Tools:** Python, scikit-learn	Single risk score with 'cannot assess' rule
Week 12	**Which signals actually matter**	• Retrain the model with one signal removed at a time (no radar, no temperature, and so on). • Run these with C and record how much each signal is really contributing. **Tools:** scikit-learn, MLflow	Ablation results table

Final two weeks — tuning and writing

Week	Focus	What to do	Done when…
Week 13	**Final tuning**	• Read C's failure analysis and fix what it exposes. • Retrain the final model, save it with its version number, and stop changing it after this week. **Tools:** PyTorch, scikit-learn, MLflow	Final model frozen
Week 14	**Write your chapter and wrap up**	• Write the 'Methodology' part of the report in simple language — what the model does and why. • Draw a clear diagram of the model pipeline. • Clean up your notebooks so someone else can run them top to bottom. • Prepare to answer the viva question: 'why anomaly detection and not normal classification?' **Tools:** draw.io / diagrams.net, Word/LaTeX, Jupyter, Git	Chapter written; notebooks clean

4. Who you depend on, and who depends on you

Person	What passes between you
Student A (Data Collection)	A gives you the clean data cube. Report any layer that looks wrong immediately — do not work around it silently.
Student C (Model Testing & Evaluation)	You give C your anomaly scores in the agreed format. C tells you where the model fails and why.
Student D (App Development)	You give D the score files so the dashboard can display them; agree the file format early.

5. Common mistakes to avoid

- Jumping straight to deep learning before trying the simple z-score baseline — the guide will ask 'is it better than the simple method?' and you must have an answer.
- Forgetting to remove the seasonal cycle, so the model flags every summer as dangerous.
- Training on all data including the 2023 event, which secretly leaks the answer and makes C's test meaningless.
- Not logging experiments, then being unable to explain in month 4 why you picked a setting.
- Reporting a low risk score for a lake that was actually hidden under cloud.

6. What you write in the final report

- Methodology — what features were built and why.
- Model design — baselines, anomaly models, temporal model, and the fusion rule.
- Ablation results — which sensors actually matter.
- Model diagrams and the 'cannot assess' logic.

Every week ends with the team stand-up: say what you finished, what is stuck, and what is next. Push your work to the shared GitHub repository the same day — never let the only copy sit on your laptop.